Fundação Getúlio Vargas

Estrutura de Dados

Relatório Lista 3

Alunos:
Ana Beatriz Botacim Rodrigues — 251708010
Larissa Conerado Ribeiro — 251708026

Professor:
Matheus Telles Werner

16 de maio de 2026

Conteúdo

1 Introdução **2**

2 Custo computacional **2**
2.1 Insert 2
2.2 RemovePlayer 2
2.3 SortByScoreInsertion 2
2.4 SortByScoreMerge 2
2.5 Formgroup 2
2.6 GetWaitingPlayers 2
2.7 PrintWaitingPlayers 3

3 Comparação experimental **3**
3.1 Descrição 3

1 Introdução

Este trabalho apresenta uma solução para um sistema de matchmaking, atribuindo e removendo jogadores de arrays, lista de espera e/ou lista do grupo, quando possível. Além de demonstrar e explicar as análises de comparação realizadas.

2 Custo computacional

Para a análise computacional considere as seguintes variáveis: s como a quantidade de jogadores no array de jogadores, g como a quantidade de jogadores solicitada para formar o grupo.

2.1 Insert

A função insert verifica a capacidade disponível no array de jogadores e, se possível, realiza a inserção. Como o índice de inserção já está pré-definido, sua complexidade é $O(1)$.

2.2 RemovePlayer

Primeiramente, a função percorre o array procurando o id do jogador, caso seja encontrado retornará true, diminuirá o tamanho em um e realocará os jogadores à direita para a posição exatamente anterior. Como percorre todo o array exatamente uma vez, tem complexidade $O(s)$.

2.3 SortByScoreInsertion

A função percorre todos os índices uma vez (até o $s-2$), e, para cada índice, verifica se o elemento seguinte está de acordo com a ordenação. Caso contrário, percorerrá os jogadores anteriores - já ordenados - fazendo trocas até esta porção do array ficar ordenada. Com isso, custo computacional é $O(s^2)$.

2.4 SortByScoreMerge

Nossa função chama outra função, `mergeSort`, que, por sua vez, é recursiva e, a cada chamada, divide o tamanho do array por dois utilizando a técnica de dividir para conquistar. Os níveis continuam aumentando até chegar ao caso base, que demora $\frac{s}{2^k} = 1 \implies k = \log_2 s$ níveis de recursão. Cada nível apresenta custo computacional total de cs, sendo c o custo de percorrer todos os elementos dentro da chamada de `merge` para fundir os subarrays na ordem correta. Não há custo adicional significativo nas chamadas recursivas, pois não são criados novos arrays durante a divisão, apenas são passados ponteiros delimitando o início e o fim de cada subarray.

Como são feitos $\log_2 s$ níveis de recursão e cada um deles custa computacionalmente algo em torno de cs, a análise desta função conclui que sua complexidade é $O(s \times \log_2 s)$

2.5 Formgroup

A estrátegia utilizada foi verificar todas as combinações possíveis de grupo, por meio da diferença entre a primeira e última pessoas em potencial, formando o primeiro grupo possível.

Para isso, o array de players foi percorrido do ínicio até o índice $s-g+1$, e, para cada um, a diferença entre o score do jogador do índice atual com o score do jogador de índice $atual+g-1$ foi computada. Caso fosse uma diferença válida, removemos os jogadores entre os considerados (inclusive) da lista de espera, chamando a função `removePlayer` g vezes.

Por fim, retornamos um ponteiro para a cópia do array de jogadores do grupo, que custa $O(g)$ para ser preenchido. Em caso negativo, apenas retornamos o solicitado. Como percorremos apenas uma vez o array de players, temos $O(s)$, e, quando formamos um grupo, gastamos $O(g+csg)$ para arrumar os arrays. Isso acarreta uma complexidade $O(s^2)$, pois $g \leq s$.

2.6 GetWaitingPlayers

A função retorna um ponteiro nulo quando o array está vazio e retorna um ponteiro para a cópia do array de jogadores, quando s difere de zero. Ao realizar a cópia do array de jogadores, acumula complexidade de $O(s)$.

2.7 PrintWaitingPlayers

A função depende da quantidade de jogadores presente no array e, por exibir cada um, seu custo computacional é de $O(s)$.

3 Comparação experimental

3.1 Descrição

Os testes foram realizados no arquivo `testes.cpp`, a explicação a seguir tem como base o código no mesmo.

Foram criados dois objetos da classe `Matchmaking`, um para testes do insertion e um para os do merge. Para usar dados aleatórios, criamos uma função que gera um array de n jogadores com score em uma distribuição uniforme de 0 a 4999. Os testes foram feitos com uma variação de 10 a 100000 jogadores. Para cada uma das 100 vezes que testamos cada tamanho de entrada em cada função de ordenação, criamos um novo array de jogadores com scores aleatórios, e, para que pudéssemos usar este mesmo array nos dois métodos, mantivemos um array base original e copiamos seus elementos, um a um, para cada objeto `Matchmaking`.

Na primeira parte do arquivo fizemos um warm-up para diminuir os erros de medição. Na fase dos testes em si, para cada tamanho, e em cada iteração, criamos um array base aleatório e os dois objetos `Matchmaking`. Em cada uma das 100 vezes para cada n, medimos o desempenho de cada uma das funções de ordenação, garantindo que elas usassem o mesmo array para ordenar, com isso, calculamos a média e usamos como métrica.

Para criar os gráficos, como sabemos que $O(n^2)$ significa que o custo computacional é algo como $c \times n^2 + O(n)$, para fazer a comparação entre o insertion sort e sua curva teórica, decidimos "normalizar" a constante c: isso foi feito dividindo a curva de $O(n^2)$ pela razão entre o $f(100000)$ – o tempo necessário para nossa função computar um array de 100000 jogadores – e 100000^2. Desta forma, os resultados ficaram muito próximos ao esperado. Caso não tivéssemos feito esse processo, a curva de complexidade real ficaria esmagada no gráfico perto de zero ao ser comparada com a curva de n^2, por conta da pequenez de c. O mesmo passo a passo foi realizado na visualização do merge sort, e, de novo, o resultado foi satisfatório, seguindo, com grande precisão, a complexidade prevista teoricamente.

Como a complexidade do insertion sort cresce muito mais rápido que a do merge, não era possível verificar se a tendência era seguida quando comparávamos ambas juntas, portanto decidimos apresentar 3 visualizações. Para gerar as representações visuais, usamos um arquivo `.ipynb`, que julgamos não ser necessário para a entrega, por simplesmente carregar os dados de `resultados.csv` e gerar gráficos simples.

Quantidade de jogadores	10	100	1000	2000	5000	10000	15000	20000	30000	40000	50000	80000	100000
Insertion (microsseg)	0	24	4085	20186	130476	563764	1141111	2097716	4425676	7784816	11698127	28446805	43442950
Merge (microsseg)	5	31	535	1136	3308	7755	9828	13286	18530	26540	33150	57360	74069

Tabela 1: Performance dos algoritmos para a quantidade de jogadores adicionada

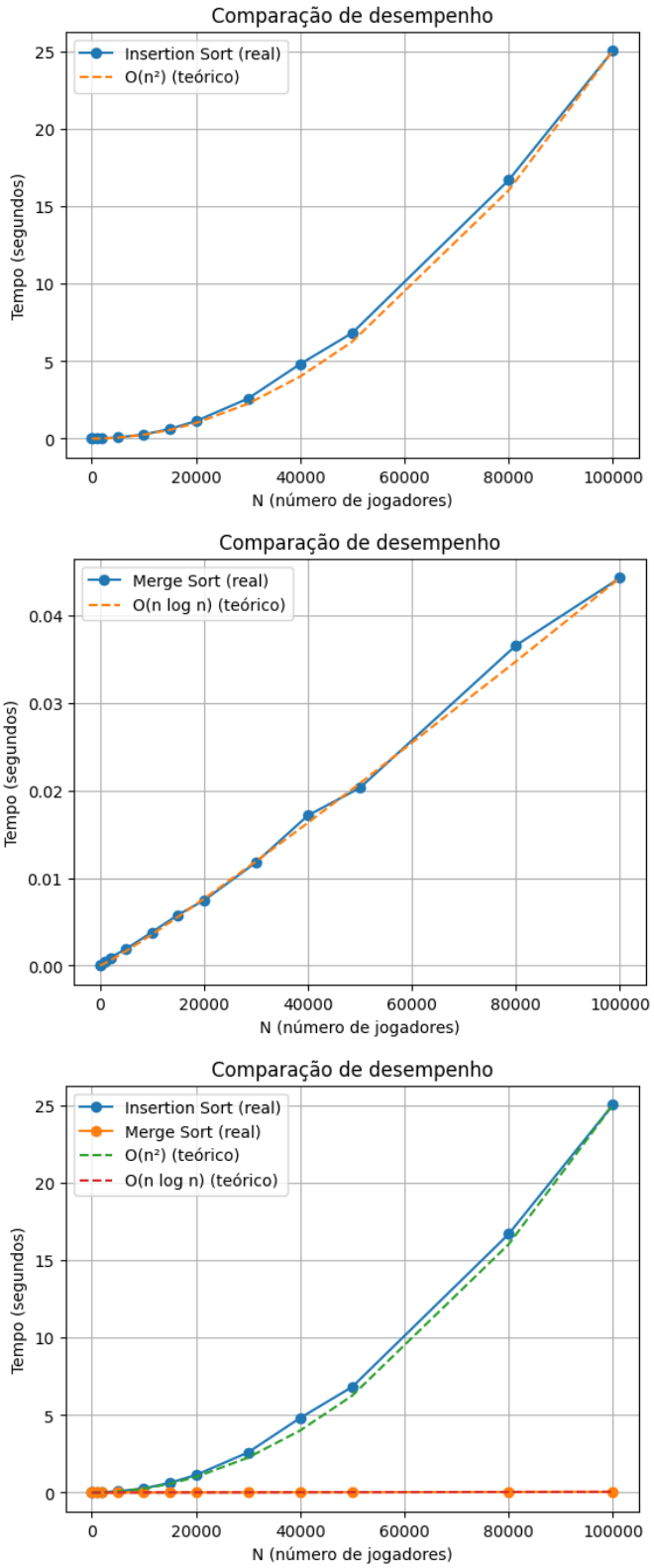

Figura 1: Comparação de desempenho dos algoritmos